

Especificação do Projeto

Plataforma de Live Streaming Distribuída

ACH2147 – Desenvolvimento de Sistemas de Informação Distribuídos

Grupo 1:
Lucas Leão Ferreira Barbosa
Lion Chen
João Victor de Pascale Souza
Giovanni Willik del Piccolo

Escola de Artes, Ciências e Humanidades – EACH – USP
Prof. Renan Alves

29 de junho de 2026

Sumário

1	Introdução	2
1.1	Objetivos	2
2	Fundamentação Teórica	2
2.1	Modelos de Sistemas Distribuídos (Cap. 1)	2
2.2	Transparência de Distribuição (Cap. 1)	3
2.3	Escalabilidade (Cap. 1)	3
2.4	Estilos Arquiteturais (Cap. 2)	3
2.5	Comunicação (Cap. 4)	4
2.5.1	Conexões Temporárias e Duradouras	4
2.6	Coordenação e Ordenação (Cap. 5)	4
2.6.1	Ordenação de Mensagens no Chat	4
2.6.2	Exclusão Mútua via Fila	4
2.7	Nomeação (Cap. 6)	5
2.8	Consistência e Replicação (Cap. 7)	5
2.9	Tolerância a Falhas (Cap. 8)	5
2.9.1	Modelo de Falhas Assumido	5
2.9.2	Classificação de Falhas por Componente	6
2.9.3	Redundância	6
2.10	Processos e Threads (Cap. 3)	6
2.11	Virtualização e Containers (Cap. 3)	6
3	Arquitetura do Sistema	7
3.1	Visão Geral dos Componentes	7
3.2	Diagrama da Arquitetura	8
3.3	Tipos de Mensagens e Formatos	8
4	Diagramas de Sequência	8
4.1	Fluxo 1 – Streamer Inicia uma Live	8
4.2	Fluxo 2 – Espectador Assiste a uma Live	9
4.3	Fluxo 3 – Chat em Tempo Real	10
4.4	Fluxo 5 – Recuperação do Orchestrator	10
5	Tolerância a Falhas – Detalhamento	11
5.1	Healthcheck dos Workers	11
5.2	Healthcheck do Orchestrator	11
5.3	Failover do Banco de Dados	12

1 Introdução

Este documento apresenta a especificação técnica de uma plataforma de **transmissão de vídeo ao vivo (*live streaming*) distribuída**, desenvolvida no contexto da disciplina ACH2147 – Desenvolvimento de Sistemas de Informação Distribuídos. O sistema é inspirado em plataformas como Twitch e YouTube Live, construído sobre um conjunto de microsserviços desacoplados orquestrados via Docker Compose.

O **streamer** transmite vídeo a partir do OBS Studio via protocolo RTMP. O sistema ingere esse fluxo, transcodifica para HLS sob demanda (um *worker* dedicado por live) e distribui o vídeo para múltiplos **espectadores** através de um player web. Em paralelo, cada live possui um **chat em tempo real** baseado em Pub/Sub com Redis.

1.1 Objetivos

- Permitir que múltiplas lives ocorram simultaneamente, cada uma isolada em seu próprio worker.
- Escalar horizontalmente a transcodificação (1 worker por live, criado dinamicamente).
- Garantir **tolerância a falhas**: workers e o orquestrador podem cair e o sistema se recupera automaticamente.
- Manter os **dados críticos** (usuários, lives, doações/metadados) consistentes e replicados.
- Oferecer **chat em tempo real** com alta disponibilidade.

2 Fundamentação Teórica

2.1 Modelos de Sistemas Distribuídos (Cap. 1)

O projeto combina três modelos clássicos:

1. **Cliente-Servidor em camadas (*multitier*)**: Frontend → API/Chat → Banco/Redis.
2. **Baseado em eventos (*event-based*)**: O Ingest Service publica eventos na fila; o Orchestrator reage – desacoplamento temporal e referencial.
3. **Orientado a serviços (SOA/microsserviços)**: cada serviço (Ingest, API, Chat, Worker Pool) é independente, com responsabilidade única e implantável isoladamente.

O modelo predominante é o **orientado a eventos com estilo de microsserviços**, pois a criação de lives é inerentemente assíncrona e reativa.

2.2 Transparência de Distribuição (Cap. 1)

O sistema implementa múltiplas formas de transparência:

Tabela 1: Tipos de transparência implementados.

Tipo	Implementação no Projeto
Acesso	Espectadores acessam HLS via HTTP sem conhecer a existência de workers internos
Localização	O espectador não sabe em qual container estão os segmentos de vídeo
Migração	Ao recriar um worker falho, a URL HLS permanece a mesma
Falha	Se um worker cai, o orquestrador recria transparentemente sem impacto ao espectador
Replicação	PostgreSQL Primary + Réplica é acessado como instância única pela API

2.3 Escalabilidade (Cap. 1)

- **Escalabilidade de tamanho:** cada nova live resulta em um novo worker – escala horizontal linear.
- **Escalabilidade geográfica:** fora do escopo (seria resolvida com CDN geo-distribuída).
- **Escalabilidade administrativa:** sistema dentro de um único domínio administrativo (Docker host).

2.4 Estilos Arquiteturais (Cap. 2)

- **Arquitetura em camadas:** interface (frontend) → processamento (API/orquestrador) → dados (PostgreSQL).
- **Publish-Subscribe:** Redis Pub/Sub para chat e RabbitMQ para eventos de ciclo de vida.
- **Pipeline (*pipe-and-filter*):** OBS → nginx-rtmp → FFmpeg → Volume HLS → Espectador.

2.5 Comunicação (Cap. 4)

Tabela 2: Classificação dos tipos de comunicação utilizados.

Tipo	Modalidade	Onde no projeto
Transitória síncrona	Request-response	HTTP REST (Frontend $\leftrightarrow$ API); GET de chunks HLS
Transitória assíncrona	Streaming	WebSocket (chat bidirecional); RTMP (OBS $\rightarrow$ Ingest)
Persistente assíncrona	Message queue	RabbitMQ: mensagens sobrevivem crashes

Toda a stack utiliza **TCP**, pois a integridade dos dados é mais importante que latência mínima: RTMP/TCP (ingestão), HTTP/TCP (HLS), AMQP/TCP (fila), WebSocket/TCP (chat). UDP foi descartado pois perda de chunks de vídeo ou de mensagens de chat é inaceitável para a experiência do usuário.

2.5.1 Conexões Temporárias e Duradouras

- **Duradouras:** RTMP (enquanto a live durar), WebSocket (enquanto o espectador estiver no chat), conexões do worker com a fila.
- **Temporárias:** chamadas REST e cada GET de chunk HLS (comunicação transitente).

2.6 Coordenação e Ordenação (Cap. 5)

2.6.1 Ordenação de Mensagens no Chat

Para garantir que todas as mensagens do chat sejam exibidas na mesma ordem para todos os espectadores, o Chat Service atribui um **timestamp NTP** a cada mensagem recebida antes de publicá-la no Redis. Isso funciona como um **relógio físico centralizado** (o servidor de chat é a referência temporal), garantindo uma **ordenação total** das mensagens dentro de cada canal.

2.6.2 Exclusão Mútua via Fila

O RabbitMQ serializa as requisições de criação de lives, implementando implicitamente um mecanismo de **coordenação centralizada**: o Orchestrator processa uma mensagem por vez, evitando conflitos de alocação de recursos (portas, containers).

2.7 Nomeação (Cap. 6)

Tabela 3: Esquema de nomeação dos recursos do sistema.

Recurso	Identificador	Tipo de Nomeação
Live	token (UUID)	Plana (identificador puro)
Worker	worker_id (container)	Plana
Canal de chat	chat:live_<token>	Estruturada (hierárquica)
Stream HLS	/streams/live_<token>/index.m3u8	Estruturada (hierárquica)
Usuário	user_id (PK Postgres)	Plana
Serviços	api, redis, rabbitmq	DNS interno Docker

Resolução de nomes: O DNS interno do Docker Compose resolve nomes de serviço para IPs de container. O token da live resolve para o worker correspondente via mapa em memória do Orchestrator (cache) e, de forma autoritativa, via SELECT no PostgreSQL.

2.8 Consistência e Replicação (Cap. 7)

O sistema aplica o **Teorema CAP** de forma consciente, utilizando os **Dados críticos (CP)**: PostgreSQL Primary + Réplica opera com **consistência forte**. Se a rede falhar entre primary e réplica, a transação é recusada (sacrifica disponibilidade). Nunca duplica ou perde dados de usuários/lives.

A replicação do PostgreSQL é do tipo **primary-backup** com failover automático: se o Primary cai, a Réplica assume como novo Primary.

2.9 Tolerância a Falhas (Cap. 8)

2.9.1 Modelo de Falhas Assumido

O sistema assume o modelo **fail-stop**: componentes que falham param completamente e essa parada é detectável via healthcheck (timeout). Não são consideradas falhas bizantinas.

2.9.2 Classificação de Falhas por Componente

Tabela 4: Classificação e tratamento de falhas.

Componente	Tipo de Falha	Mecanismo de Recuperação
Worker (FFmpeg)	Crash (parada)	Orchestrator detecta via heart-beat; respawn automático
Orchestrator	Crash (parada)	Health Monitor detecta; redeploy + recuperação via PostgreSQL
PostgreSQL Primary	Crash (parada)	Réplica assume automaticamente (failover)
Redis	Crash (parada)	Mensagens em vôo perdidas; novas conexões reconectam
RabbitMQ	Crash (parada)	Filas duráveis: mensagens persistem em disco

2.9.3 Redundância

- **Redundância de informação:** replicação PostgreSQL Primary → Réplica.
- **Redundância física:** Podemos ter redundância física deployando os containers em máquinas fisicamente diferentes, configurando uma maior resiliência da nossa aplicação.
- **Redundância temporal:** heartbeats periódicos permitem detecção e re-execução.

2.10 Processos e Threads (Cap. 3)

- **Chat Service:** utiliza Async I/O (event loop) com suporte a threads para lidar com centenas de conexões WebSocket simultâneas de forma eficiente.
- **Workers FFmpeg:** processos CPU-bound isolados em containers (paralelismo por processo, não por thread).
- **Orchestrator:** uma thread principal consome a fila RabbitMQ; threads secundárias gerenciam os healthchecks (uma thread por worker monitorado).
- **API (FastAPI/Uvicorn):** workers assíncronos atendem requests concorrentes.

2.11 Virtualização e Containers (Cap. 3)

O Docker é o **alicerce** do projeto, utilizado como camada de **virtualização a nível de SO**:

- Cada serviço roda em container isolado (isolamento de recursos e falhas).
- O Orchestrator cria/destrói workers dinamicamente via Docker SDK a partir de uma imagem base + ENV token.
- Docker Compose orquestra todo o ambiente multi-container.
- Volumes Docker persistem os chunks HLS entre container e host.

3 Arquitetura do Sistema

3.1 Visão Geral dos Componentes

Tabela 5: Componentes do sistema e suas responsabilidades.

Componente	Tecnologia	Responsabilidade	Estado
Ingest Service	nginx-rtmp	Recebe stream RTMP do OBS; publica evento {token da live} na fila	Stateless
API Service	FastAPI	REST: GET /lives, POST /users; persiste no Postgres	Stateless
RabbitMQ	AMQP	Fila de eventos “nova live” (desacoplamento temporal)	Stateful (durável)
Orchestrator	Python + Docker SDK	Consome fila; sobe 1 worker/live; monitora heartbeat; respawn	Stateful efêmero (cache)
Worker	FFmpeg	Transcodifica RTMP → HLS (.m3u8 + .ts)	Stateless (descartável)
Health Monitor	Python	Monitora o Orchestrator; redeploy + recuperação no crash	Stateless
Chat Service	FastAPI + WebSocket	Recebe e faz broadcast de mensagens; timestamp NTP	Stateful de sessão (socket)
Redis	Pub/Sub	Distribui mensagens do chat por canal/live	Stateful (mensagens em voo)
PostgreSQL Primary	PostgreSQL	Fonte da verdade: usuários, lives, metadados	Stateful autoritativo
PostgreSQL Réplica	PostgreSQL	Standby; assume no failover	Stateful autoritativo
Volume HLS	Docker Volume	Buffer de chunks de vídeo entre worker e espectador	Stateful (durável)
Frontend	HTML + JS + hls.js	Player de vídeo + cliente WebSocket do chat (roda no cliente)	Stateless

A decisão de projeto é manter **toda a lógica stateless** e isolar o estado em quatro backends de dados: PostgreSQL, Redis, RabbitMQ e Volume HLS – o que habilita escala horizontal e recuperação simplificada.

3.2 Diagrama da Arquitetura

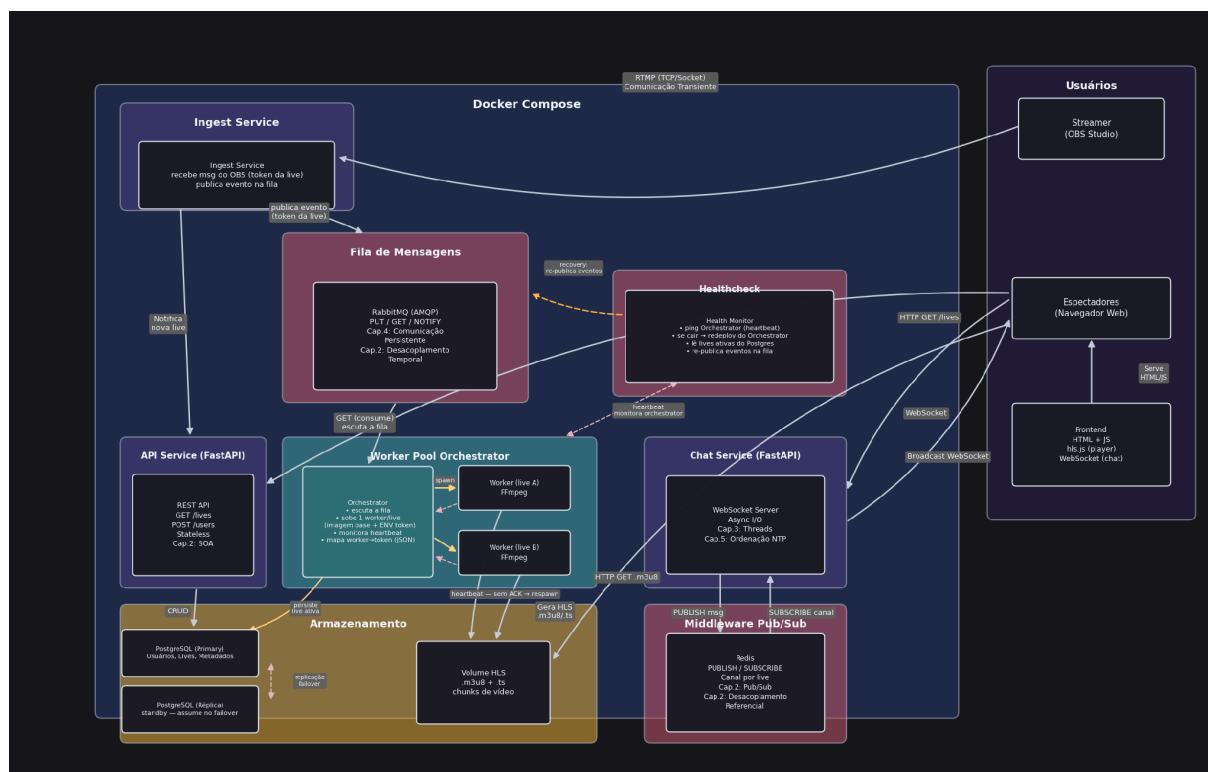


Figura 1: Arquitetura geral do sistema.

3.3 Tipos de Mensagens e Formatos

Tabela 6: Mensagens trocadas no sistema.

Mensagem	Origem → Destino	Formato
live.started { token }	Ingest → RabbitMQ	JSON
POST /lives, POST /users	Frontend/Ingest → API	JSON (HTTP body)
Chunks de vídeo	Worker → Volume HLS → Frontend	Binário .ts + playlist .m3u8
chat:live_X { user, msg, ts }	Chat → Redis → Chat	JSON
Heartbeat	Worker ↔ Orchestrator ↔ Health Monitor	Ping/ACK (curto)

4 Diagramas de Sequência

4.1 Fluxo 1 – Streamer Inicia uma Live

O fluxo começa quando o streamer conecta o OBS Studio ao Ingest Service via RTMP. O servidor de borda recebe essa notificação, manda de maneira assíncrona os dados da nova live para a API (que persiste no PostgreSQL) e registra a requisição no RabbitMQ. O Orchestrator consome essa mensagem e cria um novo cluster de containers para aquela live: um responsável pela conexão persistente RTMP com o OBS e outro que transcodifica o vídeo via FFmpeg, gerando segmentos HLS no volume compartilhado.

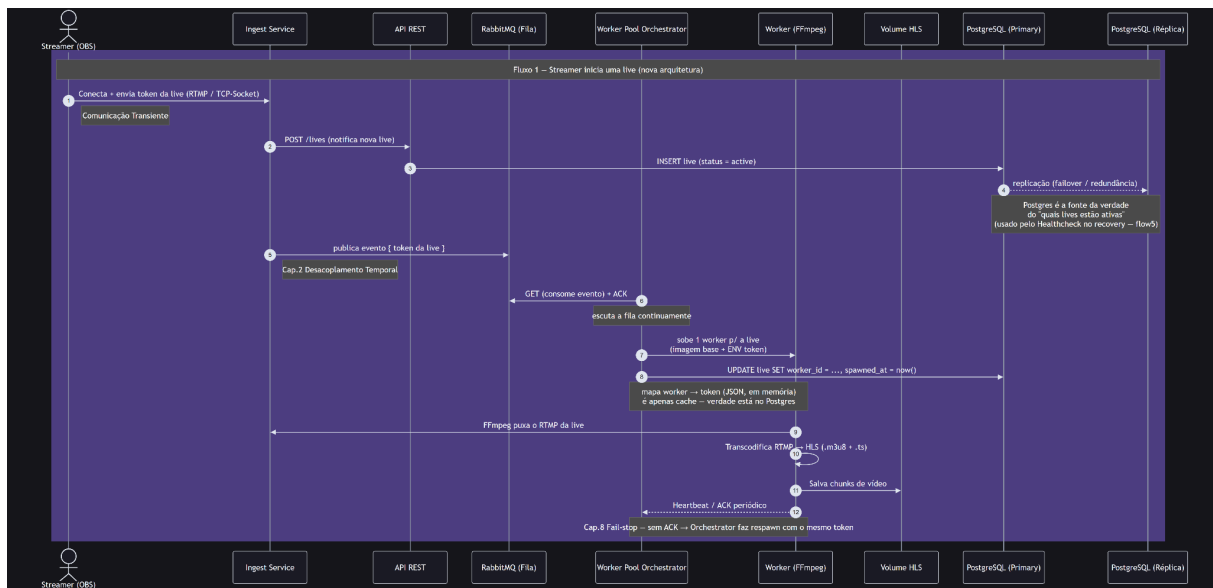


Figura 2: Streamer inicia uma live (ingest → fila → orchestrator → worker → HLS).

4.2 Fluxo 2 – Espectador Assiste a uma Live

O espectador acessa o frontend, consulta a API para listar lives ativas e, ao selecionar uma, realiza requests HTTP periódicos ao nginx para obter os chunks HLS do volume compartilhado. O player hls.js no navegador gerencia automaticamente o download progressivo dos segmentos .ts referenciados na playlist .m3u8.

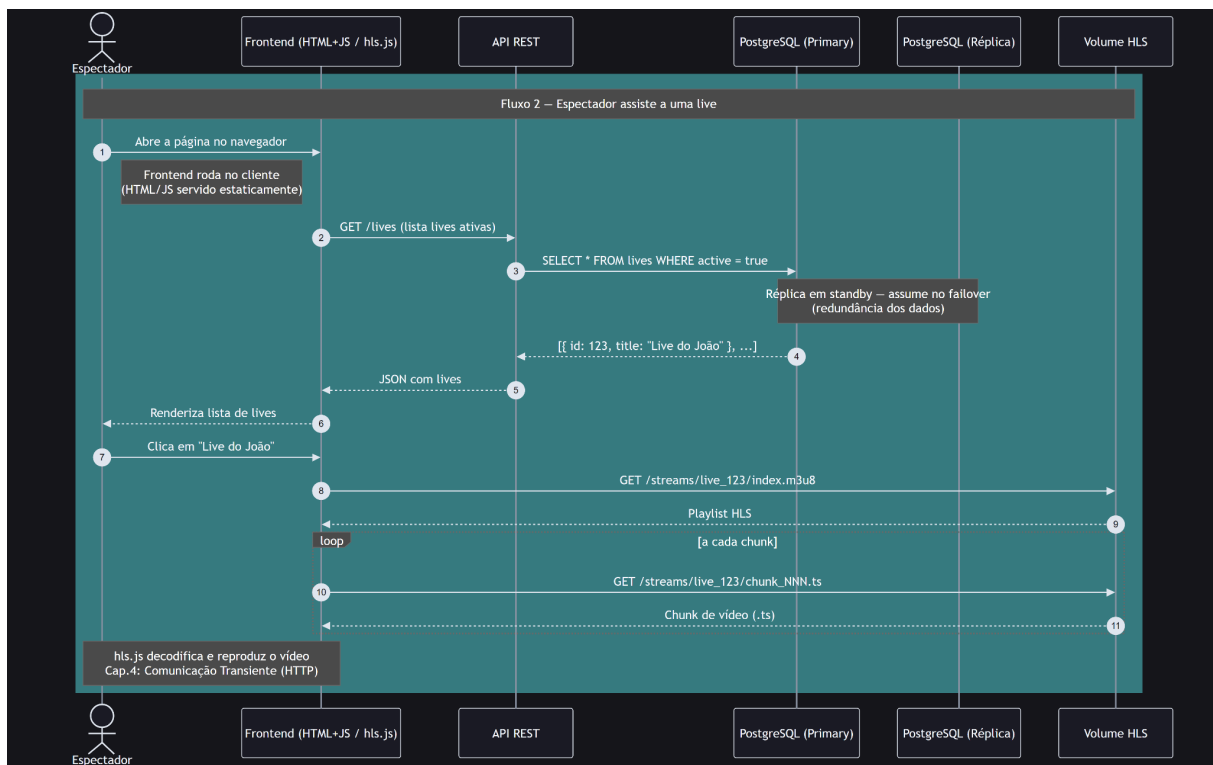


Figura 3: Espectador assiste a uma live (frontend → API/HLS → Volume).

4.3 Fluxo 3 – Chat em Tempo Real

Ao entrar em uma live, o espectador abre uma conexão WebSocket com o Chat Service, que o inscreve no canal Redis chat:live_<token>. Mensagens enviadas são recebidas pelo Chat Service, recebem um timestamp NTP para ordenação, e são publicadas no Redis Pub/Sub. Todos os subscribers daquele canal recebem a mensagem em tempo real.

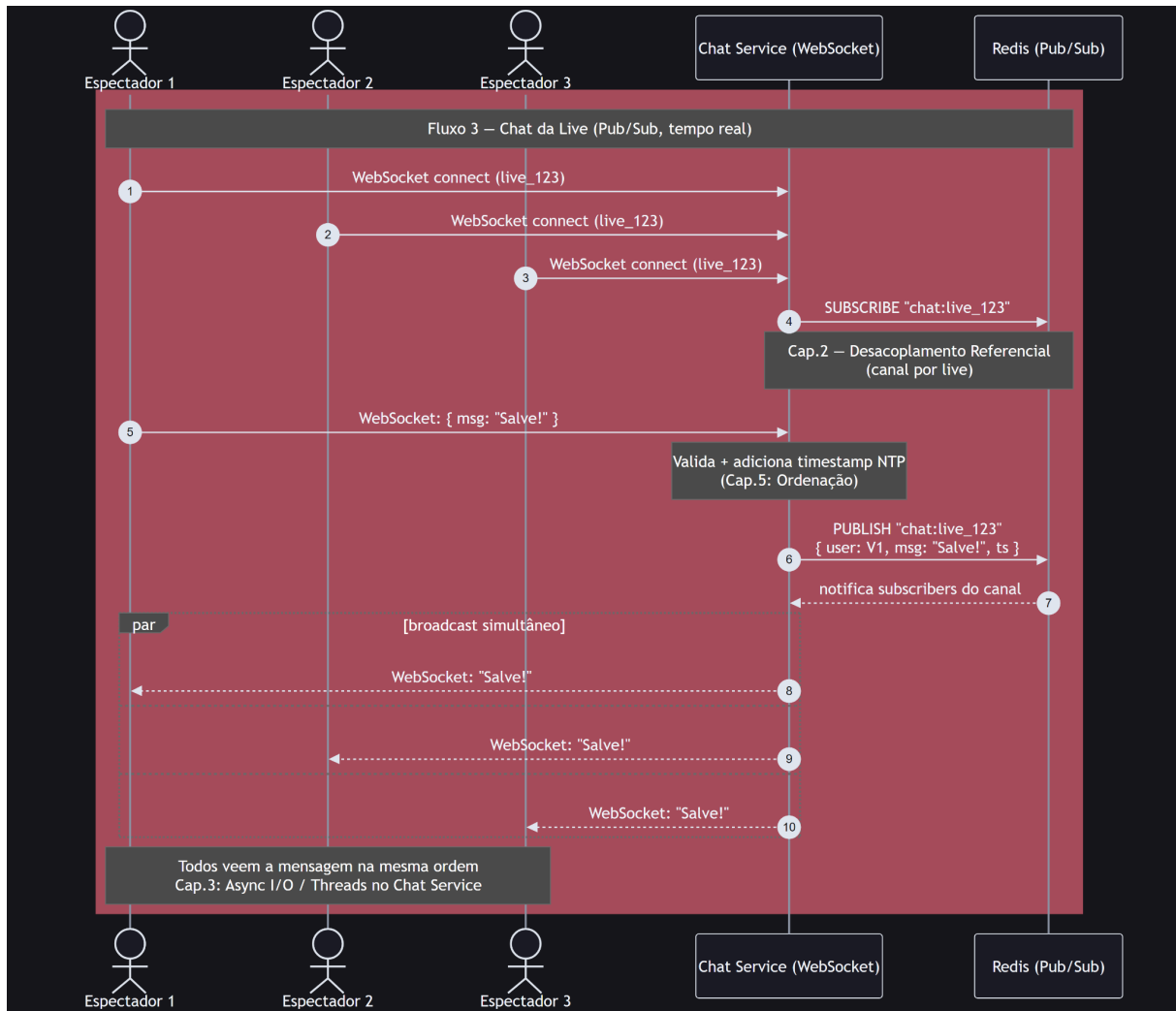


Figura 4: Chat em tempo real via WebSocket + Redis Pub/Sub.

4.4 Fluxo 5 – Recuperação do Orchestrator

Quando o Health Monitor detecta que o Orchestrator parou de responder (modelo *fail-stop*), ele inicia o processo de recuperação: busca os metadados de todas as lives online no PostgreSQL, cria novamente outro Orchestrator via Docker, e coloca no RabbitMQ todas as lives que devem ser reiniciadas.

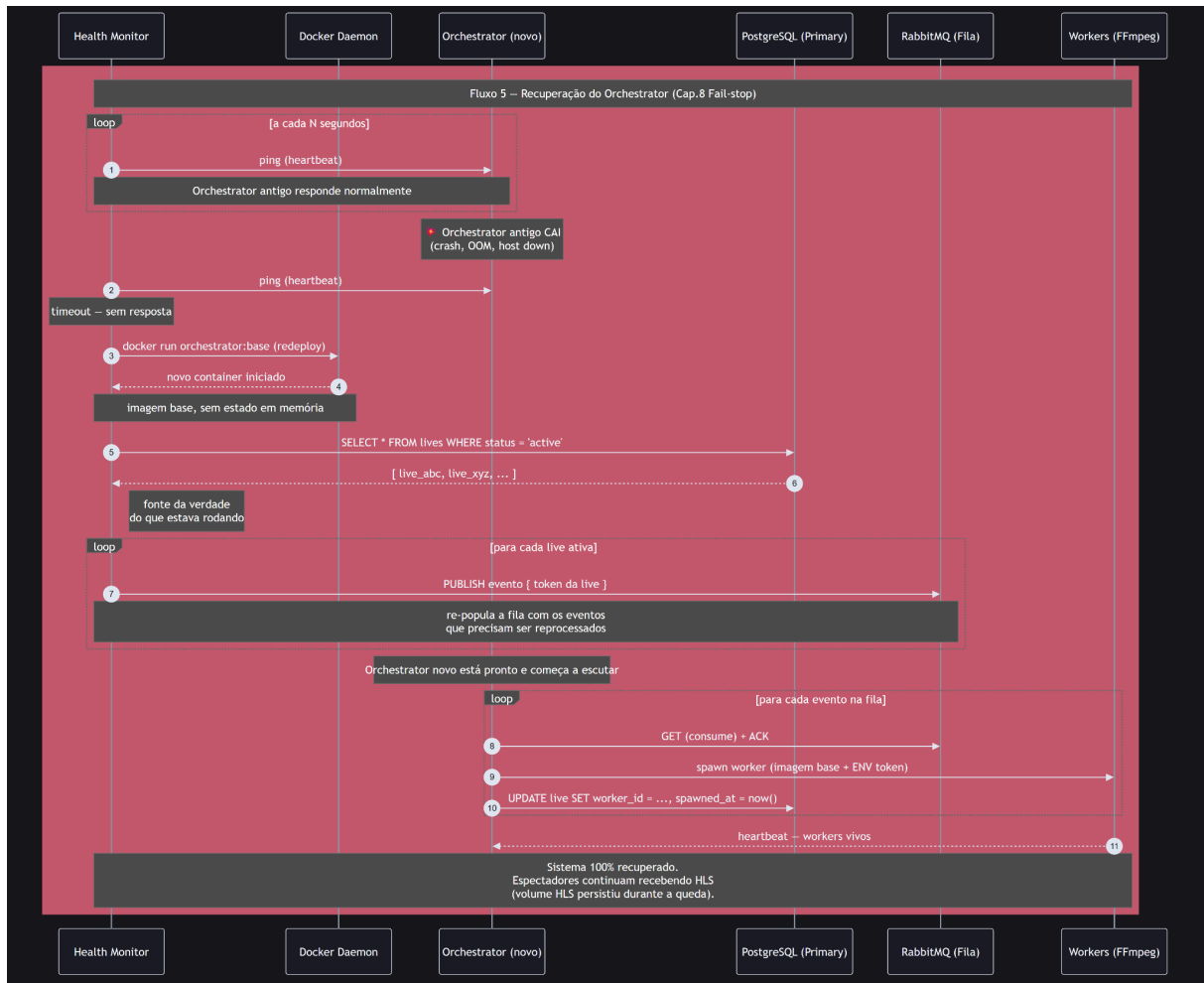


Figura 5: Recuperação do Orchestrator via Health Monitor (Cap. 8 – Fail-stop).

5 Tolerância a Falhas – Detalhamento

5.1 Healthcheck dos Workers

O Orchestrator foi codificado em python e utiliza o Event Loop para lidar com a concorrência, ou seja, um Orchestrator pode cuidar de mais de um worker, enviando heartbeats periódicos:

1. Orchestrator envia PING ao worker via endpoint HTTP interno.
2. Worker responde ACK se estiver operacional.
3. Se nenhum ACK for recebido dentro do timeout Δt , o Orchestrator considera o worker como falho.
4. O Orchestrator destrói o container antigo e cria um novo worker com o mesmo token da live, reconectando ao stream RTMP.

5.2 Healthcheck do Orchestrator

O Health Monitor é um serviço que roda em um **container separado** do Orchestrator:

1. Health Monitor envia PING periódico ao Orchestrator.
2. Caso não receba ACK dentro do timeout, assume *crash*.
3. Faz **redploy** do Orchestrator a partir da **imagem base** (sem estado em memória).
4. Novo Orchestrator lê do **Postgres** as lives com status='active' (somente as ativas).
5. **Re-publica** os eventos correspondentes no RabbitMQ.
6. O novo Orchestrator consome a fila e **recria os workers** (imagem base + ENV token), atualizando o Postgres.
7. O **Volume HLS** persistiu durante a queda, então os espectadores praticamente não percebem a interrupção.

5.3 Failover do Banco de Dados

O PostgreSQL opera com replicação **streaming replication** (Primary → Réplica). O failover é automático: se o Primary cai, a Réplica assume como novo Primary. A API reconecta automaticamente ao novo endpoint.